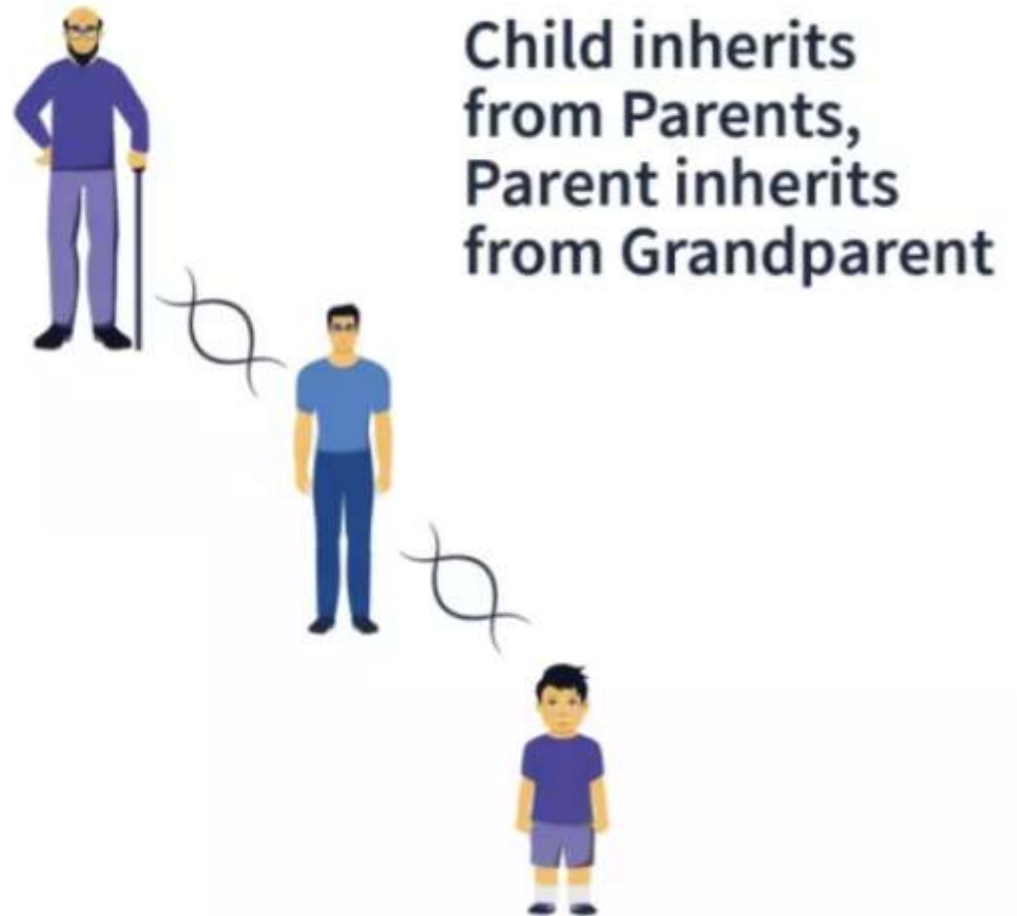
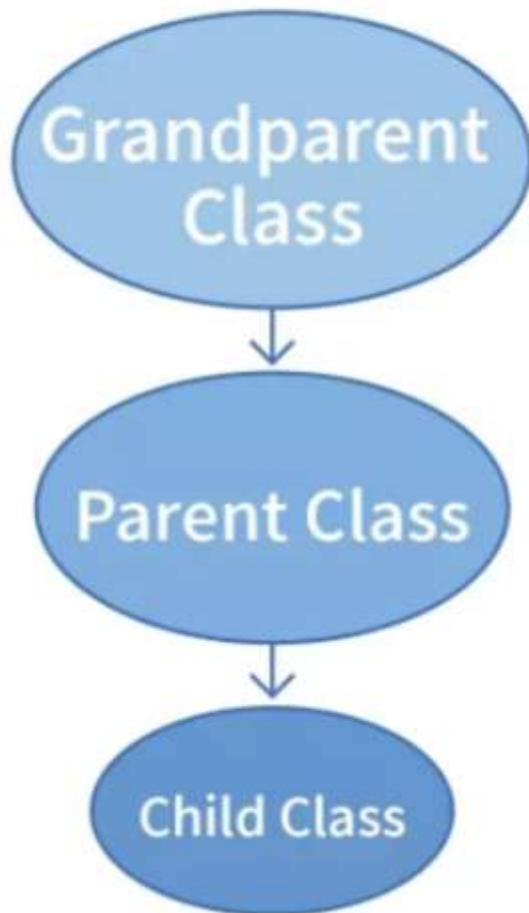

Nhắc lại kiến thức bài trước

Kế thừa

Kế thừa

- Kế thừa?



Mối quan hệ kế thừa

- Lớp con và lớp cha có tính tương đồng
 - Lớp SinhVien kế thừa từ lớp Ngươi.
 - Một sinh viên là một người

Lớp cha
Ngươi



Lớp con SinhVien

Các thuộc tính &
phương thức của
lớp cha Ngươi

thuộc tính và
phương thức riêng

Cú pháp (Python)

- Cú pháp (Python)
<lớp con> (<lớp cha>)

```
class Nguoi:  
    name = ""  
    age = 0  
  
class SinhVien(Nguoi):  
    student_id = 0
```

Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
name, age
studentId

Cú pháp (Java)

- Cú pháp (Java)

<lớp con> extends <lớp cha>

```
class Nguoi {  
    String name; int age;  
}  
  
class SinhVien extends Nguoi {  
    int studentId;  
}
```

Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
name, age
studentId

Cú pháp (C++, C#)

- Cú pháp (C++)
<lớp con> : <lớp cha>

```
#include <string>
class Nguoi {
    std::string name; int age;
};

class SinhVien : Nguoi {
    int studentId;
};
```

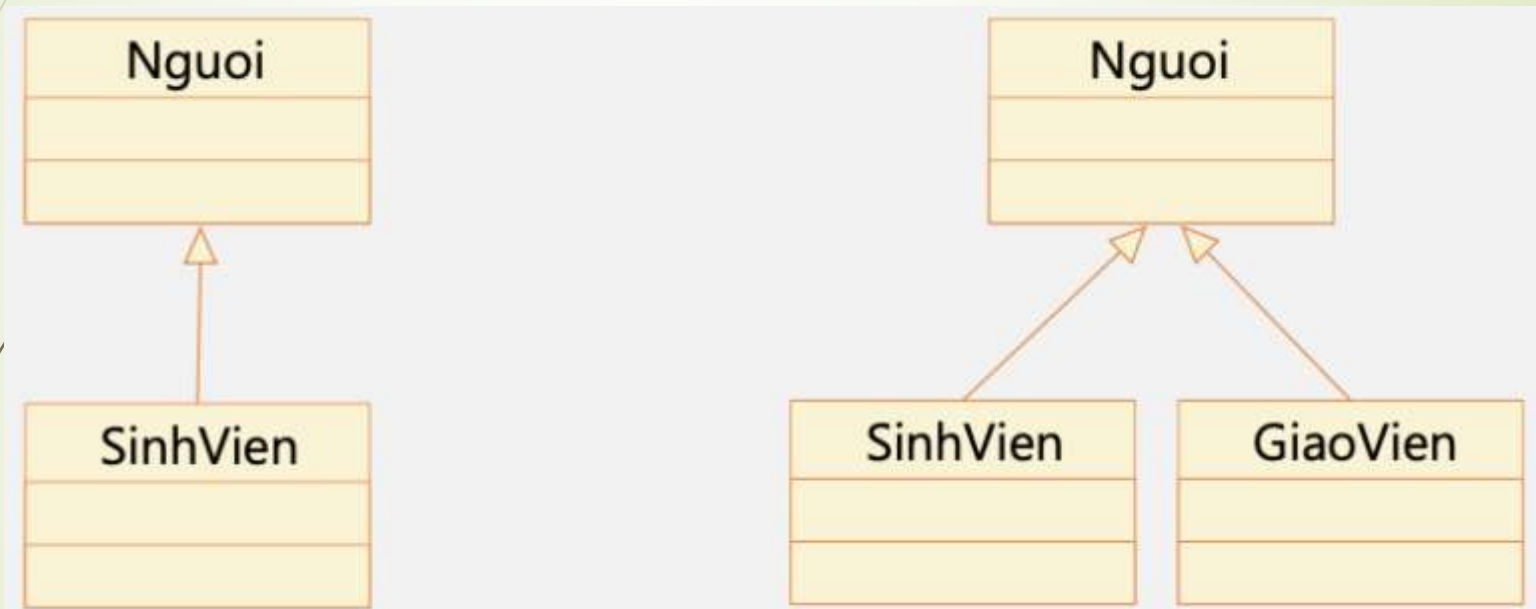
Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
name, age
studentId

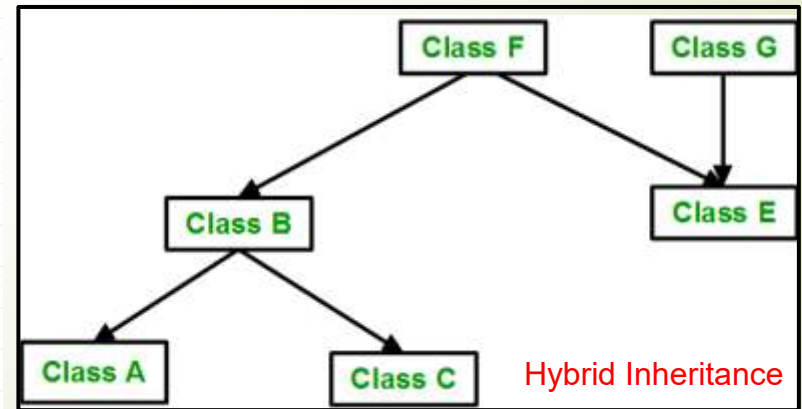
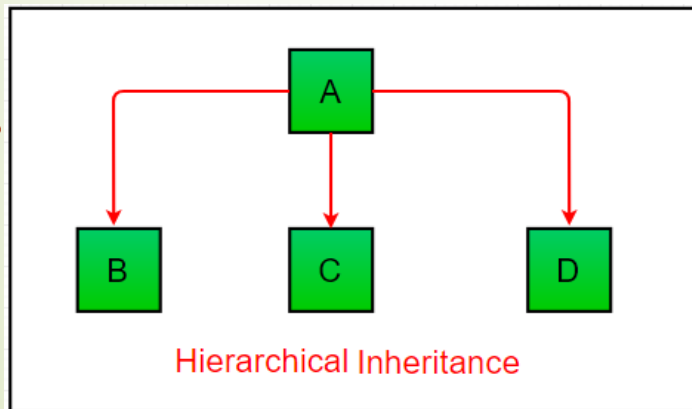
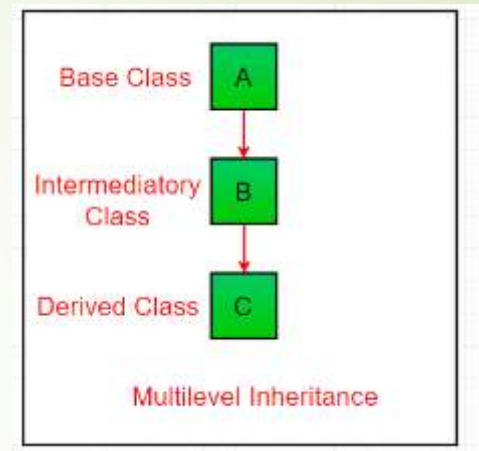
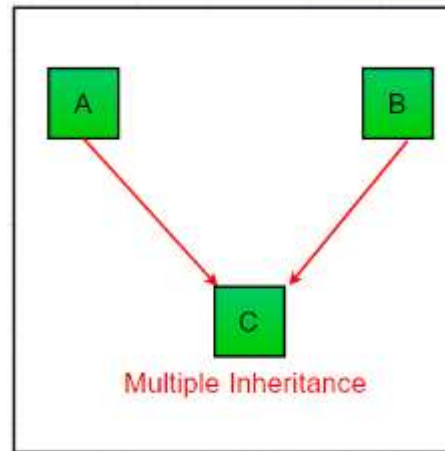
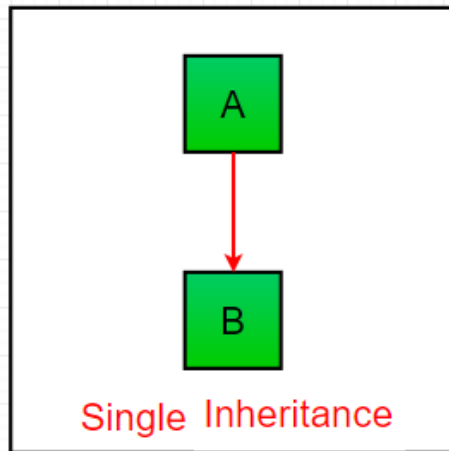
Cây phân cấp kế thừa

- Dùng mũi tên với tam giác rỗng ở đầu



Các loại kế thừa

- Các loại kế thừa trong Python:



Nguyên lý kế thừa

- Lớp con có thể thừa kế được gì từ lớp cha?
 - Kế thừa được các thành viên được khai báo là **public** và **protected** của lớp cha.
 - Không kế thừa được các thành viên **private**.
- Thành viên **protected** trong lớp cha được truy cập trong:
 - Các thành viên lớp cha
 - Các thành viên lớp con

Gọi phương thức của lớp cha

- Tái sử dụng các đoạn mã của lớp cha trong lớp con
- Gọi phương thức khởi tạo (sử dụng từ khóa **super**):
 - Bắt buộc nếu lớp cha không có phương thức khởi tạo mặc định
 - Phải được khai báo tại dòng lệnh đầu tiên trong phương thức khởi tạo của lớp con
- Gọi các phương thức của lớp cha

Python:

```
super().__init__(name, age)  
super().say_hello()
```

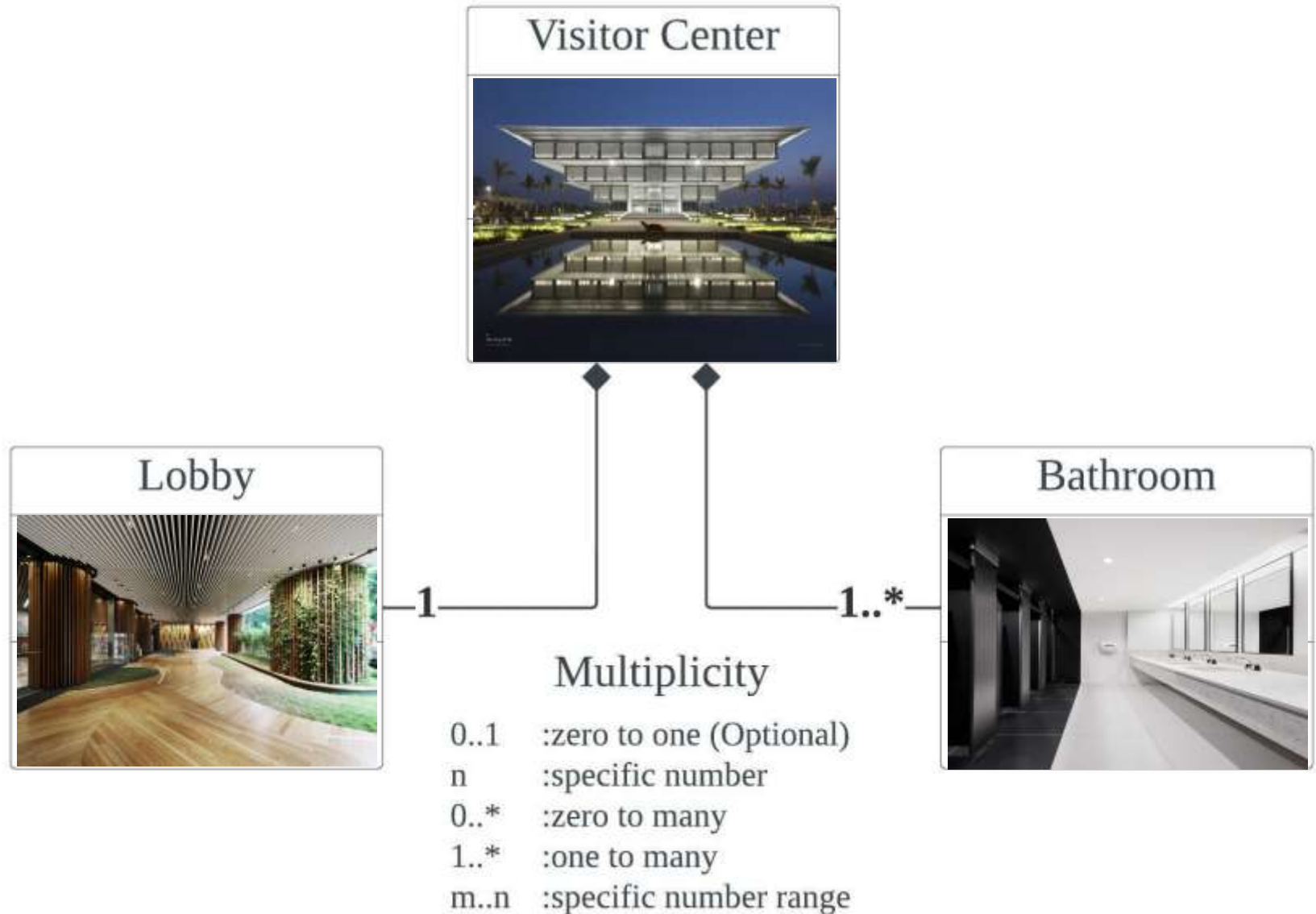
Java:

```
super(name, age);           // trong constructor  
super.sayHello();          // gọi method lớp cha
```

C++:

```
Person(name, age)          // trong danh sách khởi tạo  
Person::sayHello();        // gọi method lớp cha
```

Quan hệ thành phần



Roadmap môn học — Bạn đang ở đâu?

Tuần 1



- Giới thiệu OOP
- 4 tính chất
- OOAD
- Cài đặt IDE

Tuần 2



- Lập trình căn bản
- Quy tắc định danh
- Ôn tập

Tuần 3



- Trừu tượng hóa
- Xây dựng lớp cơ bản
- Tạo và sử dụng đối tượng Class/ Object

Tuần 4



- Đóng gói
- Hàm khởi tạo
- Nạp chồng phương thức
- Quan hệ Kết Tập

Tuần 5



- Kế thừa
- Các nguyên lý kế thừa
- Quan hệ kế thừa, quan hệ thành phần

▶ BẠN ĐANG Ở ĐÂY

Tuần 6

- Đa hình
- Magic methods
- Ngoại lệ
- Tổng hợp

Tuần 7

- File I/O nâng cao
- Dict nâng cao
- Tuần tự hóa đối tượng

Tuần 8

- UML
- Các biểu đồ cơ bản
- Phân tích & Thiết kế HĐT

4 tính chất OOP

1

Trừu tượng hóa

Abstraction

2

Đóng gói

Encapsulation

3

Kế thừa

Inheritance

4

Đa hình

Polymorphism

OOP06: Đa hình & Ngoại lệ

Polymorphism & Exception Handling — Mở rộng & Nâng cao



Nội dung

1. Đa hình & Lớp trừu tượng
 2. Magic methods & Operator overloading
 3. Ngoại lệ & Đóng gói nâng cao
 4. Ví dụ tổng hợp
- 

1

Đa hình & Lớp trừu tượng

Polymorphism & Abstract Base Class

Đa hình

Đa hình (Polymorphism) là gì?

Đa hình (Polymorphism) xuất phát từ tiếng Hy Lạp: poly = nhiều, morph = dạng.

Trong lập trình hướng đối tượng, đa hình là khả năng một phương thức có CÙNG TÊN nhưng thể hiện HÀNH VI KHÁC NHAU

tùy thuộc vào đối tượng cụ thể đang gọi nó. Đây là tính chất thứ 4 và cuối cùng trong 4 tính chất OOP.

Ví dụ thực tế giúp hiểu đa hình

Hãy nghĩ đến hành động "phát ra âm thanh" của các loài động vật:

- Con chó → sủa "Gâu gâu"
- Con mèo → kêu "Meo meo"
- Con vịt → kêu "Quạc quạc"

Cùng một hành động "phát ra âm thanh", nhưng mỗi loài thể hiện KHÁC NHAU.

Đây chính là đa hình: cùng giao diện (interface), khác cài đặt (implementation).

```
# Minh họa bằng Python:  
class Cho:  
    def phat_am(self):  
        return "Gâu gâu"
```

```
class Meo:  
    def phat_am(self):  
        return "Meo meo"
```

```
class Vit:  
    def phat_am(self):  
        return "Quạc quạc"
```

```
# Đa hình: cùng 1 vòng lặp, 3 hành vi  
for con in [Cho(), Meo(), Vit()]:  
    print(con.phat_am())
```

Đa hình

Trong Python, đa hình xuất hiện ở 5 dạng:

1. Đa hình trong hàm tích hợp

len(), print(),
abs() ... hoạt
động với nhiều
kiểu

2. Đa hình trong hàm tự định nghĩa

Hàm ta viết nhận
được nhiều kiểu
đối tượng

3. Đa hình với các phương thức lớp

Nhiều class cùng
có phương thức
giống tên

4. Đa hình và kế thừa

Lớp con override
phương thức lớp
cha

5. Đa hình với 1 hàm và nhiều đối tượng

1 hàm xử lý danh
sách chứa nhiều
loại object

Chúng ta sẽ lần lượt xét từng dạng với ví dụ cụ thể

1.1. Đa hình trong hàm tích hợp (Built-in functions)

- Các hàm tích hợp sẵn của Python như `len()`, `print()`, `abs()`, `type()`, `sorted()`... được thiết kế để hoạt động với NHIỀU kiểu dữ liệu khác nhau.
- Cùng một hàm, nhưng hành vi thay đổi tùy thuộc vào kiểu đối tượng truyền vào. Đây là dạng đa hình cơ bản nhất — chúng ta đã dùng hàng ngày mà không nhận ra!

```
# Hàm len() — hoạt động với nhiều kiểu:
print(len("Python"))      # 6      (đếm ký tự)
print(len([1, 2, 3, 4]))  # 4      (đếm phần tử)
print(len({"a":1, "b":2}))# 2      (đếm cặp key-value)
print(len((10, 20, 30)))  # 3      (đếm phần tử tuple)
```

```
# Hàm print() — in được mọi kiểu:
print(42)                 # int → "42"
print(3.14)               # float → "3.14"
print("Hello")            # str → "Hello"
print([1, 2, 3])          # list → "[1, 2, 3]"
```

```
# Hàm abs() — giá trị tuyệt đối:
print(abs(-5))            # 5
print(abs(3.14))          # 3.14
print(abs(3 + 4j))        # 5.0 (độ dài số phức!)
```

```
# Toán tử + cũng là đa hình:
print(2 + 3)              # 5      (cộng số)
print("ab" + "cd")        # "abcd" (nối chuỗi)
print([1,2] + [3,4])      # [1,2,3,4] (nối list)
```

```
# Cùng toán tử +, hành vi khác nhau
# tùy kiểu dữ liệu → ĐA HÌNH!
```

➡ Bạn đã dùng đa hình hàng ngày mà không biết!

`len()`, `print()`, `abs()`, toán tử + đều là đa hình — cùng tên/ký hiệu nhưng xử lý khác nhau tùy kiểu dữ liệu.

Sau này ở phần Magic Methods, ta sẽ hiểu CƠ CHẾ đằng sau: `len(x)` thực chất gọi **`x.__len__()`**

1.2. Đa hình trong hàm tự định nghĩa

Khi ta tự viết hàm, nếu hàm đó chỉ dựa vào các phương thức/toán tử đa hình bên trong, thì hàm ta viết cũng tự động trở thành đa hình — nhận được nhiều kiểu dữ liệu khác nhau mà không cần kiểm tra kiểu.

```
# Hàm tự định nghĩa — đa hình tự nhiên:
def cong(a, b):
    """Hàm này hoạt động với BẤT KỲ kiểu nào
    hỗ trợ toán tử +"""
    return a + b

# Gọi với nhiều kiểu khác nhau:
print(cong(5, 3))           # 8 (int + int)
print(cong(2.5, 1.5))       # 4.0 (float + float)
print(cong("Xin ", "chào")) # "Xin chào" (str + str)
print(cong([1,2], [3,4]))   # [1,2,3,4] (list + list)
```

```
# Ví dụ 2: Hàm nhân đôi
def nhan_doi(x):
    return x * 2

print(nhan_doi(5))          # 10
print(nhan_doi("Ha"))       # "HaHa"
print(nhan_doi([1, 2]))     # [1, 2, 1, 2]

# Ví dụ 3: Hàm lấy phần tử đầu
def phan_tu_dau(collection):
    return collection[0]

print(phan_tu_dau("Python")) # "P"
print(phan_tu_dau([10,20]))   # 10
print(phan_tu_dau((5,6,7)))   # 5
```

➡ Hàm tự định nghĩa trở thành đa hình khi bên trong nó dùng các **phép toán/phương thức đa hình**. Ta KHÔNG CẦN kiểm tra kiểu (if isinstance...) — Python tự xử lý. Đây gọi là Duck Typing.

1.3. Đa hình với các phương thức lớp

Khi nhiều lớp KHÔNG CÓ quan hệ kế thừa với nhau nhưng cùng định nghĩa phương thức có CÙNG TÊN, ta có thể gọi phương thức đó trên các đối tượng khác nhau theo cùng một cách. Python sẽ tự gọi đúng phiên bản phương thức của lớp tương ứng. Đây là Duck Typing — "nếu nó kêu như vịt, thì nó là vịt".

```
class Cho:
    def __init__(self, ten):
        self.ten = ten
    def tieng_keu(self):
        return f"{self.ten}: Gâu gâu!"
    def so_chan(self):
        return 4

class Meo:
    def __init__(self, ten):
        self.ten = ten
    def tieng_keu(self):
        return f"{self.ten}: Meo meo!"
    def so_chan(self):
        return 4

class Vit:
    def __init__(self, ten):
        self.ten = ten
    def tieng_keu(self):
        return f"{self.ten}: Quạc quạc!"
    def so_chan(self):
        return 2
```

```
# 3 class KHÔNG kế thừa nhau
# nhưng cùng có tieng_keu() và so_chan()

a = Cho("Rex")
b = Meo("Kitty")
c = Vit("Donald")

# Đa hình: duyệt list gọi cùng method
for con in [a, b, c]:
    print(con.tieng_keu())

# Rex: Gâu gâu!
# Kitty: Meo meo!
# Donald: Quạc quạc!

# Mỗi đối tượng tự biết cách "kêu"
# → cùng tên phương thức, khác hành vi
# → ĐA HÌNH!
```

➡ Cho, Meo, Vit không kế thừa nhau nhưng cùng có `tieng_keu()` → gọi được trong 1 vòng lặp.
Đây là Duck Typing: Python không kiểm tra kiểu, chỉ kiểm tra "có phương thức không".

1.4. Đa hình và kế thừa (Method Overriding)

Khi lớp con định nghĩa phương thức có CÙNG TÊN với lớp cha, phương thức lớp con "ghi đè" phương thức lớp cha. Python tự động chọn đúng phiên bản dựa trên kiểu THỰC TẾ của đối tượng tại thời điểm chạy (late binding). Đây là dạng đa hình QUAN TRỌNG NHẤT và phổ biến nhất trong OOP.

```
class HìnhHoc:
    def tinh_dien_tich(self):
        return 0 # Mặc định

class HìnhTron(HìnhHoc): # Kế thừa
    def __init__(self, r):
        self.r = r
    def tinh_dien_tich(self): # Override!
        return 3.14159 * self.r ** 2

class HìnhChuNhat(HìnhHoc): # Kế thừa
    def __init__(self, a, b):
        self.a, self.b = a, b
    def tinh_dien_tich(self): # Override!
        return self.a * self.b
```

```
# Hàm nhận đối tượng lớp CHA
# nhưng chạy phương thức lớp CON:
def in_thong_tin(hinh):
    ten = type(hinh).__name__
    s = hinh.tinh_dien_tich()
    print(f"{ten}: S = {s:.2f}")
```

```
in_thong_tin(HìnhTron(5))
# HìnhTron: S = 78.54
```

```
in_thong_tin(HìnhChuNhat(3, 4))
# HìnhChuNhat: S = 12.00
```

```
# Python gọi đúng phiên bản override
# dựa trên kiểu THỰC TẾ → late binding
```

➡ Khác với dạng 1.3 (không kế thừa), ở đây các lớp CÓ quan hệ cha-con. Hàm `in_thong_tin(hinh)` khai báo nhận kiểu `HìnhHoc`, nhưng chạy đúng phương thức của `HìnhTron` hay `HìnhChuNhat`. Đây là sức mạnh cốt lõi của OOP: viết code tổng quát, hoạt động đúng với mọi lớp con.

1.5. Đa hình với một hàm và nhiều đối tượng

Đây là ứng dụng thực tế mạnh nhất của đa hình: ta có MỘT hàm duy nhất, truyền vào một danh sách chứa NHIỀU loại đối tượng khác nhau. Hàm xử lý tất cả mà KHÔNG CẦN BIẾT cụ thể từng đối tượng thuộc lớp nào. Đây chính là lý do OOP mạnh hơn lập trình thủ tục.

```
class HìnhTron(HìnhHoc):
    def __init__(self, r): self.r = r
    def tinh_dien_tich(self):
        return 3.14159 * self.r ** 2

class HìnhChuNhat(HìnhHoc):
    def __init__(self, a, b): self.a, self.b = a, b
    def tinh_dien_tich(self):
        return self.a * self.b

class HìnhTamGiac(HìnhHoc):
    def __init__(self, a, h): self.a, self.h = a, h
    def tinh_dien_tich(self):
        return 0.5 * self.a * self.h
```

```
# 1 HÀM — xử lý TẤT CẢ loại hình:
def tong_dien_tich(ds_hinh):
    tong = 0
    for h in ds_hinh:
        tong += h.tinh_dien_tich()
    return tong

# Danh sách chứa NHIỀU loại đối tượng:
ds = [HìnhTron(5),
      HìnhChuNhat(3, 4),
      HìnhTamGiac(6, 8)]

print(f"Tổng S = {tong_dien_tich(ds):.2f}")
# Tổng S = 114.54

# Thêm hình mới? Chỉ tạo class mới,
# KHÔNG SỬA hàm tong_dien_tich!
```

➡ Đây là đỉnh cao của đa hình — 1 hàm `tong_dien_tich()` hoạt động với MỌI loại hình. Khi thêm `HìnhBìnhHanh` mới, chỉ cần tạo class + override `tinh_dien_tich()`, KHÔNG sửa code cũ. Nhưng vấn đề: nếu **QUÊN** override `tinh_dien_tich()` thì sao? → Cần ABC!

Lớp trừu tượng — Abstract Base Class (ABC)

Khái niệm Lớp trừu tượng

- Lớp trừu tượng là lớp KHÔNG THỂ tạo đối tượng trực tiếp, chỉ làm "bản thiết kế" cho các lớp con. Nó chứa phương thức trừu tượng (@abstractmethod): chỉ khai báo tên, lớp con BẮT BUỘC override. Và phương thức thường (concrete): có cài đặt đầy đủ, lớp con kế thừa dùng ngay hoặc override.
- Trong Python: kế thừa ABC từ module abc, đánh dấu @abstractmethod. Tương tự interface trong Java.

```
from abc import ABC, abstractmethod

class HìnhHoc(ABC):
    @abstractmethod
    def tinh_dien_tich(self):
        pass
    @abstractmethod
    def tinh_chu_vi(self):
        pass
    def hien_thi(self): # Concrete — dùng ngay
        print(f"S = {self.tinh_dien_tich():.2f}")

# h = HìnhHoc() → TypeError! Không tạo được!

class HìnhTron(HìnhHoc):
    def __init__(self, r): self.r = r
    def tinh_dien_tich(self): # BẮT BUỘC
        return 3.14159 * self.r ** 2
    def tinh_chu_vi(self): # BẮT BUỘC
        return 2 * 3.14159 * self.r
```

Tại sao cần ABC?

- Ép buộc lớp con override → tránh quên
- Thiết kế "hợp đồng" rõ ràng
- Ngăn tạo đối tượng vô nghĩa (HìnhHoc() trừu tượng quá)
- Kết nối: Trừu tượng hóa + Kế thừa + Đa hình = ABC

Nếu quên override?

```
class HìnhSai(HìnhHoc):
    def tinh_dien_tich(self): ...
    # thiếu tinh_chu_vi!
hs = HìnhSai() → TypeError!
```

➔ ABC = mắt xích kết nối Trừu tượng hóa + Kế thừa + Đa hình

2

Magic Methods & Operator overloading

Các phương thức đặc biệt:

`__str__`, `__eq__`, `__lt__`, `__add__`

Magic Methods (Dunder Methods) là gì?

- Magic methods (hay dunder methods — double underscore) là các phương thức đặc biệt có tên `__***__`.
- Bạn đã biết `__init__` từ tuần 2 — đó là 1 magic method! Python tự gọi nó khi tạo đối tượng.
- Python tự gọi magic methods khi ta dùng `print()`, `==`, `<`, `+`, `sorted()`, `len()`...
- Magic methods chính là CƠ CHẾ ĐA HÌNH cốt lõi: `print(x)` gọi `x.__str__()`, `a+b` gọi `a.__add__(b)`.
- Mỗi class override magic methods → cùng toán tử, khác hành vi = ĐA HÌNH!

Bức tranh tổng thể: Các nhóm Magic Methods trong Python

Nhóm	Magic Methods	Công dụng
Khởi tạo & Hủy	<code>__init__</code> , <code>__del__</code> , <code>__new__</code>	Tạo, hủy, kiểm soát tạo đối tượng
Biểu diễn chuỗi	<code>__str__</code> , <code>__repr__</code> , <code>__format__</code>	<code>print()</code> , <code>repr()</code> , f-string
So sánh	<code>__eq__</code> , <code>__ne__</code> , <code>__lt__</code> , <code>__gt__</code> , <code>__le__</code> , <code>__ge__</code>	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>sorted()</code>
Toán học	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> , <code>__truediv__</code> , <code>__mod__</code> , <code>__pow__</code>	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>
Toán gán	<code>__iadd__</code> , <code>__isub__</code> , <code>__imul__</code>	<code>+=</code> , <code>-=</code> , <code>*=</code>
Unary	<code>__neg__</code> , <code>__pos__</code> , <code>__abs__</code> , <code>__invert__</code>	<code>-x</code> , <code>+x</code> , <code>abs(x)</code> , <code>~x</code>
Container	<code>__len__</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>__contains__</code> , <code>__iter__</code>	<code>len()</code> , <code>[]</code> , <code>in</code> , <code>for</code>
Hash & Bool	<code>__hash__</code> , <code>__bool__</code>	<code>set()</code> , dict key, <code>if obj</code>
Context Mgr	<code>__enter__</code> , <code>__exit__</code>	<code>with</code> statement
Callable	<code>__call__</code>	<code>obj()</code> — gọi object như hàm

Do giới hạn chương trình, chúng ta sẽ tập trung vào các nhóm: Biểu diễn, So sánh, Toán học, Container

__str__ và __repr__ - Hiển thị đối tượng

`__str__`: chuỗi cho NGƯỜI DÙNG — dễ đọc, thân thiện. Gọi bởi `print()` và `str()`.

`__repr__`: chuỗi cho LẬP TRÌNH VIÊN — chính xác, copy-paste tạo lại được. Gọi bởi `repr()` và khi hiển thị trong list.

```
class SinhVien:
    def __init__(self, ten, msv, diem):
        self.ten = ten
        self.msv = msv
        self.diem = diem

    def __str__(self):
        """Cho người dùng – dễ đọc"""
        return f"SV: {self.ten} ({self.msv})"

    def __repr__(self):
        """Cho developer – tái tạo được"""
        return (f"SinhVien('{self.ten}', "
                f"'{self.msv}', {self.diem})")
```

```
sv = SinhVien("Nguyen Van A", "SV001", 8.5)

# Không có __str__:
# <__main__.SinhVien object at 0x7f...>

# Có __str__:
print(sv)          # SV: Nguyen Van A (SV001)
print(repr(sv))
# SinhVien('Nguyen Van A', 'SV001', 8.5)
print([sv])
# [SinhVien('Nguyen Van A', 'SV001', 8.5)]

# Quy tắc: nếu chỉ viết 1 → viết __repr__
# print() dùng __str__, fallback → __repr__
```

➡ `print()` gọi `__str__()` trên MỌI đối tượng — str, int, list, SinhVien... Cùng hàm print, khác kết quả.

Đây chính là đa hình trong hàm tích hợp đã nói ở trên — và đây là cơ chế đằng sau!

__eq__, __lt__, __hash__ - So sánh & Sắp xếp

__eq__: toán tử == (mặc định: so sánh địa chỉ bộ nhớ → vô nghĩa). __lt__: toán tử < (cần cho sorted(), min(), max()).

__hash__: hash code (cần cho set() loại trùng và dùng object làm dict key).

Ba methods này giúp object tích hợp hoàn toàn vào hệ sinh thái Python (sorted, set, dict...)

```
class SinhVien:
    # ... (__init__, __str__ như trên)
    def __eq__(self, other):
        """So sánh bằng theo mã SV"""
        if not isinstance(other, SinhVien):
            return NotImplemented
        return self.msv == other.msv

    def __lt__(self, other):
        """Nhỏ hơn theo điểm → sorted()"""
        return self.diem < other.diem

    def __hash__(self):
        """Hash theo mã SV → set(), dict"""
        return hash(self.msv)
```

```
sv1 = SinhVien("A", "SV001", 8.5)
sv2 = SinhVien("B", "SV002", 7.0)
sv3 = SinhVien("C", "SV001", 9.0)
```

```
sv1 == sv3      # True (cùng MSV)
sv2 < sv1        # True (7.0 < 8.5)
sorted([sv1, sv2]) # [sv2, sv1] tự động!
min([sv1, sv2])   # sv2 (điểm thấp nhất)
set([sv1, sv3])   # {sv1} loại trùng!
```

```
# Kết nối tuần 5:
# Trước: viết hàm sap_xep_theo_diem() thủ công
# Sau: sorted(ds_sv) → Python tự làm!
```

➡ Với __eq__ + __lt__ + __hash__, đối tượng tích hợp hoàn toàn vào hệ sinh thái Python. sorted(), min(), max(), set(), dict đều hoạt động tự nhiên — không cần viết hàm riêng nữa.

Đa hình toán tử (Operator Overloading / Nạp chồng toán tử)

Nạp chồng toán tử cho phép các toán tử +, -, *, /... hoạt động với đối tượng tự định nghĩa.

Bản chất: mỗi toán tử tương ứng 1 magic method. $a + b$ gọi $a.__add__(b)$. Mỗi class cài đặt khác → ĐA HÌNH.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):          # v1 + v2
        return Vector(self.x + other.x,
                       self.y + other.y)
    def __mul__(self, scalar):         # v * 3
        return Vector(self.x * scalar,
                       self.y * scalar)
    def __abs__(self):                 # abs(v)
        return (self.x**2 + self.y**2) ** 0.5
    def __eq__(self, other):           # v1 == v2
        return self.x==other.x and self.y==other.y
    def __str__(self):
        return f"({self.x}, {self.y})"
```

```
v1 = Vector(1, 2)
v2 = Vector(3, 4)
```

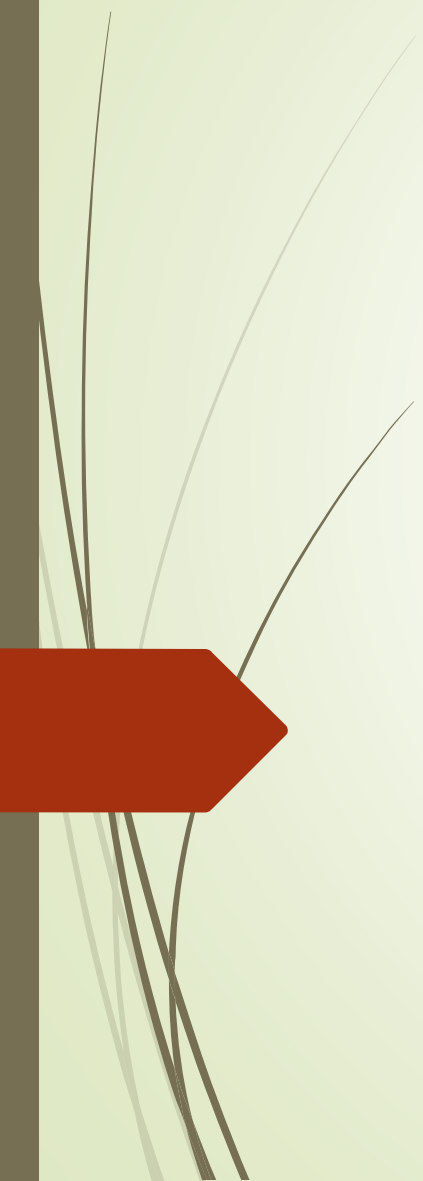
```
print(v1 + v2)      # (4, 6)
print(v1 * 3)       # (3, 6)
print(abs(v1))      # 2.236...
print(v1 == v2)     # False
```

```
# Hiểu sâu hơn:
# "abc" + "def"   → str.__add__
# [1,2] + [3,4]  → list.__add__
# v1 + v2        → Vector.__add__
# Cùng toán tử + → 3 hành vi khác nhau
# ĐÓ LÀ ĐA HÌNH!
```

Bảng ánh xạ toán tử → magic method

+	-	*	/	//	%	**	==	!=	<	>	<=	>=
__add__	__sub__	__mul__	__truediv__	__floordiv__	__mod__	__pow__	__eq__	__ne__	__lt__	__gt__	__le__	__ge__

3



Ngoại lệ

Exceptions

-
- Các chương trình rất mong manh. Sẽ thật tuyệt nếu mã luôn trả về kết quả hợp lệ, nhưng đôi khi không thể tính được kết quả hợp lệ. Chẳng hạn, không thể chia cho 0 hoặc truy cập mục thứ tám trong danh sách chỉ có năm mục.
 - Trước đây, cách duy nhất để giải quyết vấn đề này là kiểm tra nghiêm ngặt đầu vào cho mọi chức năng để đảm bảo rằng chúng có ý nghĩa.
 - Trong thế giới hướng đối tượng chúng ta không làm thế. Đó là lý do vì sao chúng ta sẽ nghiên cứu các ngoại lệ (**exceptions**), các đối tượng lỗi chỉ cần xử lý khi việc xử lý chúng là rõ ràng.

Ví dụ: raising exceptions

```
>>> print "hello world"
```

```
File "<stdin>", line 1  
print "hello world"  
      ^
```

```
SyntaxError: invalid syntax
```

```
>>> x = 5 / 0
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
ZeroDivisionError: int division or modulo by zero
```


Ví dụ: raising exceptions

```
>>> lst = [1,2,3]
>>> print(lst[3])
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

```
>>> lst + 2
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate list (not "int") to list
```

```
>>> lst.add
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'list' object has no attribute 'add'
```

Ví dụ: raising exceptions

```
>>> d = {'a': 'hello'}  
>>> d['b']
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 'b'
```

```
>>> print(this_is_not_a_var)
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'this_is_not_a_var' is not defined
```

Lỗi cú pháp và ngoại lệ

- Lỗi trong Python có hai kiểu:
 - **Syntax Error:** Còn được gọi là Parsing Errors (lỗi phân tích cú pháp), là loại lỗi cơ bản nhất, được đưa ra khi Python parser (trình phân tích cú pháp của Python) không thể hiểu được một dòng code cụ thể nào đó.
 - **Exception:** Là những lỗi mà được phát hiện trong khi chương trình đang thực thi, ví dụ như lỗi ZeroDivisionError – Lỗi chia cho 0.

Lỗi cú pháp và ngoại lệ

- Sự khác biệt giữa Lỗi cú pháp và Ngoại lệ
 - **Lỗi cú pháp:** Như tên cho thấy lỗi này là do sai cú pháp trong mã. Nó dẫn đến việc chấm dứt chương trình.

```
# initialize the amount variable
amount = 10000

# check that You are eligible to
# purchase Dsa Self Paced or not
if(amount > 2999)
    print("You are eligible to purchase Dsa Self Paced")
```

```
File "/home/ac35380186f4ca7978956ff46697139b.py", line 4
    if(amount>2999)
        ^
SyntaxError: invalid syntax
```

Lỗi cú pháp và ngoại lệ

- Sự khác biệt giữa Lỗi cú pháp và Ngoại lệ
 - **Lỗi ngoại lệ:** Các ngoại lệ được đưa ra khi chương trình đúng về mặt cú pháp, nhưng mã dẫn đến lỗi. Lỗi này không dừng việc thực thi chương trình, tuy nhiên, nó làm thay đổi quy trình bình thường của chương trình.

```
# initialize the amount variable
marks = 10000

# perform division with 0
a = marks / 0
print(a)
```

```
Traceback (most recent call last):
  File "/home/f3ad05420ab851d4bd106ffb04229907.py", line 4, in <module>
    a=marks/0
ZeroDivisionError: division by zero
```

Ngoại lệ tích hợp sẵn

Trong Python, có một số ngoại lệ tích hợp có thể được nêu ra khi xảy ra lỗi trong quá trình thực thi chương trình. Các ngoại lệ tích hợp thường gặp:

Các ngoại lệ tích hợp thường gặp

Exception	Ví dụ
ValueError	<code>int("abc")</code>
TypeError	<code>"abc" + 5</code>
IndexError	<code>[1,2][5]</code>
KeyError	<code>d['xyz']</code>
ZeroDivisionError	<code>10 / 0</code>
FileNotFoundError	<code>open('x.txt')</code>
AttributeError	<code>obj.xyz</code>

Bắt ngoại lệ

- Các câu lệnh “**try** và **except**” được sử dụng để bắt và xử lý các ngoại lệ trong Python. Các câu lệnh có thể đưa ra các ngoại lệ được giữ bên trong mệnh đề **try**, và các câu lệnh xử lý ngoại lệ được viết bên trong mệnh đề **except**.

Ví dụ 1: Khối lệnh try

- Khối lệnh **try** giúp chúng ta kiểm tra đoạn code trong **try** có xảy ra exception hay không. Nếu có exception thì exception sẽ được chuyển đến khối lệnh **except** để xử lý.
- Ví dụ 1: Hãy thử truy cập phần tử mảng có chỉ số nằm ngoài giới hạn và xử lý ngoại lệ tương ứng.

```
# Python program to handle simple runtime error
```

```
a = [1, 2, 3]
```

```
try:
```

```
    print ("Second element = %d" %(a[1]))
```

```
    # Throws error since there are only 3 elements in array
```

```
    print ("Fourth element = %d" %(a[3]))
```

```
except:
```

```
    print ("An error occurred")
```

```
-----  
Kết quả:
```

```
-----  
Second element = 2
```

```
An error occurred
```


Ví dụ 1: Khối lệnh try

- Python cho phép chúng ta bắt nhiều exception cụ thể khác nhau được ném ra từ khối lệnh try.

```
try:
    x = 1/0
    print(x)

except TypeError:
    print("Data type of variable is not suitable type!")

except ZeroDivisionError:
    print("Cannot divide by 0!")

except:
    print("An exception occurred!")
```

Kết quả:

Cannot divide by 0!

Ví dụ 2: Khối lệnh try & else

- Ví dụ 2: Chúng ta có thể sử dụng từ khóa **else** để định nghĩa một khối lệnh (block of code) sẽ được thực thi nếu không có exception nào xảy ra.

```
try:
    x = 1/1
    print(x)
except ZeroDivisionError:
    print("Cannot divide by 0!")
else:
    print("Nothing wrong.")
```

Kết quả:

1.0
Nothing wrong.

```
try:
    x = 1/0
    print(x)
except ZeroDivisionError:
    print("Cannot divide by 0!")
else:
    print("Nothing wrong.")
```

Kết quả:

Cannot divide by 0!

Ví dụ 3: Khối lệnh try & khối lệnh finally

- Python cho phép chúng ta sử dụng try với khối lệnh **finally**. Khối lệnh **finally** **luôn luôn được thực thi** bất kể khối lệnh **try** có xảy ra exception hay không.

```
try:
    x = 1/0
    print(x)
except ZeroDivisionError:
    print("Cannot divide by 0!")
finally:
    print("The 'try except' is finished!")
```

Kết quả:

```
Cannot divide by 0!
The 'try except' is finished!
```

Ví dụ 3: Khối lệnh try & khối lệnh finally

- Chúng ta thường thực hiện việc đóng các stream khi đọc/ghi file hoặc đóng các kết nối đến database trong khối lệnh **finally**.

```
#The try block will raise an error when trying to write to a read-only file
```

```
try:
```

```
    file = open("file.txt")
```

```
    try:
```

```
        file.write("Welcome to OOP!")
```

```
    except:
```

```
        print("Something went wrong when writing to the file!")
```

```
    finally:
```

```
        # always close file
```

```
        file.close()
```

```
except:
```

```
    print("Something went wrong when opening the file!")
```

```
-----
```

```
Kết quả:
```

```
-----
```

```
Something went wrong when opening the file!
```

Ví dụ 4: từ khóa `raise` exception

- Ví dụ 4: Đây là một lớp đơn giản, chỉ thêm các phần tử vào một danh sách nếu chúng là số chẵn.

```
class EvenOnly(list):
    def append(self, integer):
        if not isinstance(integer, int):
            raise TypeError("Only integers can be added")
        if integer % 2:
            raise ValueError("Only even numbers can be added")
        super().append(integer)
```

- Lớp này mở rộng một `list` (build-in list), và ghi đè phương thức `append` để kiểm tra hai điều kiện đảm bảo thành phần thêm vào là số nguyên chẵn.
- `TypeError` và `ValueError` cũng là lớp tích hợp sẵn (built-in class)
- Sử dụng `raise` để ném ra một ngoại lệ cụ thể

Ví dụ 4: từ khóa raise exception

- Ví dụ 4:

```
>>> e = EvenOnly()  
>>> e.append("a string")
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "even_integers.py", line 7, in add  
        raise TypeError("Only integers can be added")  
TypeError: Only integers can be added
```

```
>>> e.append(3)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "even_integers.py", line 9, in add  
        raise ValueError("Only even numbers can be added")  
ValueError: Only even numbers can be added
```

```
>>> e.append(2)
```

Custom Exception — Tự thiết kế ngoại lệ

- Python cho phép tự tạo ngoại lệ bằng cách KẾ THỪA từ **Exception**. Tên lỗi rõ ràng theo nghiệp vụ
- (DiemKhongHopLe rõ hơn ValueError), mang thêm dữ liệu (e.diem), phân cấp lỗi theo hệ thống.
- Kết nối OOP: custom exception KẾ THỪA từ Exception, except lớp cha bắt cả lớp con → ĐA HÌNH!

```
# Phân cấp exception theo nghiệp vụ:
class LopHocError(Exception):
    """Gốc cho mọi lỗi hệ thống"""
    pass

class DiemKhongHopLe(LopHocError):
    def __init__(self, diem):
        self.diem = diem
        super().__init__(
            f"Điểm {diem} không hợp lệ (0-10)")

class MaSVTrung(LopHocError):
    def __init__(self, msv):
        self.msv = msv
        super().__init__(f"MSV {msv} đã tồn tại")
```

```
# Sử dụng:
try:
    diem = float(input("Nhập điểm: "))
    if not 0 <= diem <= 10:
        raise DiemKhongHopLe(diem)
except DiemKhongHopLe as e:
    print(f"Lỗi: {e}")
    print(f"Giá trị sai: {e.diem}")
```

```
# Phân cấp → đa hình:
# except LopHocError → bắt CẢ HAI
#   DiemKhongHopLe và MaSVTrung
```

```
# Exception
# └─ LopHocError      ← gốc
#   └─ DiemKhongHopLe ← cụ thể
#   └─ MaSVTrung      ← cụ thể
```

➡ Custom exception = kế thừa + đa hình trong xử lý lỗi.

except LopHocError bắt được cả DiemKhongHopLe lẫn MaSVTrung → đa hình!

Ưu nhược điểm của xử lý ngoại lệ

Ưu điểm:

- **Cải thiện độ tin cậy của chương trình:** Bằng cách xử lý các ngoại lệ đúng cách, bạn có thể ngăn chương trình của mình bị lỗi hoặc tạo ra kết quả không chính xác do lỗi hoặc đầu vào không mong muốn.
- **Xử lý lỗi đơn giản hóa:** Xử lý ngoại lệ cho phép bạn tách mã xử lý lỗi khỏi logic chương trình chính, giúp dễ đọc và bảo trì mã của bạn hơn.
- **Mã sạch hơn:** Với việc xử lý ngoại lệ, bạn có thể tránh sử dụng các câu lệnh điều kiện phức tạp để kiểm tra lỗi, dẫn đến mã sạch hơn và dễ đọc hơn.
- **Gỡ lỗi dễ dàng hơn:** Khi một ngoại lệ được đưa ra, trình thông dịch Python sẽ in một dấu vết hiển thị vị trí chính xác nơi ngoại lệ xảy ra, giúp việc gỡ lỗi mã của bạn dễ dàng hơn.

Ưu nhược điểm của xử lý ngoại lệ

Nhược điểm:

- **Chi phí hoạt động:** Việc xử lý ngoại lệ có thể chậm hơn so với việc sử dụng các câu lệnh có điều kiện để kiểm tra lỗi, vì trình thông dịch phải thực hiện thêm công việc để nắm bắt và xử lý ngoại lệ.
- **Tăng độ phức tạp của mã:** Xử lý ngoại lệ có thể làm cho mã của bạn phức tạp hơn, đặc biệt nếu bạn phải xử lý nhiều loại ngoại lệ hoặc triển khai logic xử lý lỗi phức tạp.
- **Rủi ro bảo mật có thể xảy ra:** Các ngoại lệ được xử lý không đúng cách có khả năng tiết lộ thông tin hoặc tạo lỗ hổng bảo mật trong mã của bạn, vì vậy, điều quan trọng là phải xử lý các ngoại lệ một cách cẩn thận và tránh để lộ quá nhiều thông tin về chương trình của bạn.

@property — Đóng gói Pythonic

- @property là decorator cho phép truy cập thuộc tính private một cách TỰ NHIÊN như thuộc tính thường, nhưng bên trong vẫn có kiểm soát (validation). Đây là cách Python cài đặt Đóng gói (Encapsulation).
- Thay vì get_ten() / set_ten() kiểu Java → dùng obj.ten (getter) và obj.ten = 'A' (setter) kiểu Pythonic.

```
class SinhVien:
    def __init__(self, ten, msv):
        self.__ten = ten        # private
        self.__msv = msv        # private

    @property
    def ten(self):
        """Getter — chỉ đọc, không sửa được"""
        return self.__ten

    @property
    def msv(self):
        return self.__msv

sv = SinhVien("Nguyen Van A", "SV001")
print(sv.ten)        # "Nguyen Van A" ← gọi
getter
print(sv.msv)        # "SV001"
# sv.ten = "B"        # AttributeError! (không có
setter)
```

```
# KHÔNG CÓ @property:
print(sv.__ten)
# → AttributeError!
# Thuộc tính private, không truy cập được

print(sv._SinhVien__ten)
# → "Nguyen Van A"
# Name mangling — XẤU, phá vỡ đóng gói!

# CÓ @property:
print(sv.ten)
# → "Nguyen Van A"
# Truy cập tự nhiên, an toàn!

# @property = cầu nối giữa
# private (ẩn) và public (dùng)
```



@property (chỉ getter) = cho phép ĐỌC thuộc tính private mà không cho THAY ĐỔI. Đơn giản nhất!

@property - Ví dụ 2: Getter + Setter + Validation

Ví dụ 2 — Có cả getter và setter, kết hợp validation

```
class SinhVien:
    def __init__(self, ten, msv, diem):
        self.__ten = ten
        self.__msv = msv
        self.diem = diem          # Gọi setter!

    @property
    def diem(self):
        """Getter"""
        return self.__diem

    @diem.setter
    def diem(self, value):
        """Setter — kiểm tra trước khi gán"""
        if not isinstance(value, (int, float)):
            raise TypeError("Điểm phải là số")
        if not 0 <= value <= 10:
            raise DiemKhongHopLe(value)
        self.__diem = value

    @property
    def ten(self):
        return self.__ten
```

```
# Sử dụng:
sv = SinhVien("A", "SV001", 8.5)

# Đọc — gọi getter:
print(sv.diem)          # 8.5

# Ghi — gọi setter (có validation):
sv.diem = 9.0           # OK ✓
print(sv.diem)          # 9.0

# Ghi sai — setter raise exception:
try:
    sv.diem = 15         # Ngoài [0, 10]!
except DiemKhongHopLe as e:
    print(f"Lỗi: {e}")

try:
    sv.diem = "abc"      # Không phải số!
except TypeError as e:
    print(f"Lỗi: {e}")

# __init__ cũng gọi setter → validate ngay!
# sv2 = SinhVien("B", "SV002", -5) → Lỗi!
```

➡ @property + setter + raise Exception = bộ ba hoàn hảo cho đóng gói.

Dữ liệu được kiểm tra TỰ ĐỘNG mỗi khi gán — kể cả trong __init__.

So với Java: obj.diem = 9 thay vì obj.setDiem(9) → code Pythonic hơn, ngắn hơn, nhưng an toàn ngang.

@property - Ví dụ 3: Thuộc tính tính toán (Computed)

Ví dụ 3 — @property không lưu trữ, mà TÍNH TOÁN mỗi lần truy cập

```
class HìnhChuNhat:
    def __init__(self, chieu_dai, chieu_rong):
        self.chieu_dai = chieu_dai
        self.chieu_rong = chieu_rong

    @property
    def chieu_dai(self):
        return self.__chieu_dai

    @chieu_dai.setter
    def chieu_dai(self, value):
        if value <= 0:
            raise ValueError(f"Chiều dài > 0")
        self.__chieu_dai = value

    @property
    def chieu_rong(self):
        return self.__chieu_rong

    @chieu_rong.setter
    def chieu_rong(self, value):
        if value <= 0:
            raise ValueError(f"Chiều rộng > 0")
        self.__chieu_rong = value

    @property
    def dien_tich(self):
        # Computed!
        """Không lưu, tính mỗi lần truy cập"""
        return self.__chieu_dai * self.__chieu_rong

    @property
    def chu_vi(self):
        # Computed!
        return 2*(self.__chieu_dai+self.__chieu_rong)
```

```
# Sử dụng:
hcn = HìnhChuNhat(5, 3)

# Truy cập như thuộc tính bình thường:
print(hcn.chieu_dai)      # 5
print(hcn.chieu_rong)     # 3
print(hcn.dien_tich)      # 15 (tính toán!)
print(hcn.chu_vi)         # 16 (tính toán!)

# Thay đổi kích thước:
hcn.chieu_dai = 10
print(hcn.dien_tich)      # 30 (tự cập nhật!)

# Validation:
# hcn.chieu_dai = -5      → ValueError!
# hcn.dien_tich = 100     → AttributeError!
# (computed → không có setter)

# 3 loại @property:
# 1. Chỉ getter (read-only)
# 2. Getter + setter (read-write + validate)
# 3. Computed (tính toán, không lưu trữ)
```

4



Ví dụ tổng hợp

Bài toán Quản lý Hàng Hóa — Trước vs Sau

Code tuần 5 — Nhận diện các vấn đề

```
class HangHoa:
    def __init__(self, ma, ten, nsx, gia):
        self.__ma = ma
        self.__gia = gia          # private!
    def inTTin(self):
        return f"Mã: {self.__ma} | Giá: {self.__gia}"

class HangDienMay(HangHoa):
    def __init__(self, ma, ten, nsx, gia, bh):
        super().__init__(ma, ten, nsx, gia)
        self.__bh = bh
    def inTTin1(self):             # ← Tên KHÁC lớp cha!
        return f"{super().inTTin()} | BH: {self.__bh}"
```

5 vấn đề

1. print(sp) → địa chỉ bộ nhớ vô nghĩa
2. inTTin() vs inTTin1() → mất đa hình
3. sorted(ds) → TypeError!
4. Giá âm vẫn tạo được
5. Lớp con không truy cập được __gia
6. HangHoa() tạo được (vô nghĩa)

Ảnh xạ: Vấn đề → Giải pháp chương 6

Vấn đề	Giải pháp	Kiến thức
print(sp) vô nghĩa	Cài __str__	Magic method
inTTin() vs inTTin1()	Cùng tên → override	Đa hình
sorted(ds) lỗi	Cài __lt__	Magic method
Giá âm tạo được	@property + raise	Property + Custom exc
Không truy cập __gia	@property getter	@property
HangHoa() vô nghĩa	ABC + @abstractmethod	ABC

Code mới — Áp dụng kiến thức chương 6

```
from abc import ABC, abstractmethod

class GiaKhongHopLe(Exception):
    def __init__(self, gia):
        self.gia = gia
        super().__init__(f"Giá {gia} không hợp lệ")

class HangHoa(ABC):
    def __init__(self, ma, ten, nsx, gia):
        self.__ma, self.__ten, self.__nsx = ma, ten, nsx
        self.gia = gia          # Gọi setter!

    @property
    def ma_hang(self): return self.__ma
    @property
    def ten_hang(self): return self.__ten
    @property
    def gia(self): return self.__gia
    @gia.setter
    def gia(self, v):
        if v < 0: raise GiaKhongHopLe(v)
        self.__gia = v

    @abstractmethod
    def loai_hang(self): pass
    def inTTin(self):
        return (f"[{self.loai_hang()}] {self.__ma}"
                f" | {self.__ten} | {self.__gia:,.0f}đ")

    def __str__(self): return self.inTTin()
    def __eq__(self, o): return self.__ma==o.__ma
    def __lt__(self, o): return self.__gia<o.__gia
    def __hash__(self): return hash(self.__ma)
```

```
class HangDienMay(HangHoa):
    def __init__(self, ma, ten, nsx, gia, bh, dap, cs):
        super().__init__(ma, ten, nsx, gia)
        self.__bh, self.__dap, self.__cs = bh, dap, cs
    def loai_hang(self): return "Điện máy"
    def inTTin(self):      # Override → ĐA HÌNH
        return (f"{super().inTTin()}"
                f" | BH:{self.__bh}th"
                f" | {self.__dap}V | {self.__cs}W")

# Demo/Sử dụng:
ds = [HangDienMay("DM01", "Tủ lạnh", "LG",
                  12000000, 24, 220, 150),
      # HangSanhSu(...), HangThucPham(...)
]

for sp in sorted(ds):  # __lt__ → tự sắp xếp
    print(sp)           # __str__ → đa hình

# Lưu file an toàn:
with open("kho.txt", "w") as f:
    for sp in ds:
        f.write(repr(sp) + "\n")
```



1 bài toán dùng TẤT CẢ: ABC + override + __str__ + __lt__ + __eq__ + @property + custom exc + with

Tổng kết — 4 tính chất OOP đã hoàn thiện!



Trừu tượng hóa

`class, ABC, @abstractmethod`

Tuần 3



Đóng gói

`__private, @property, getter/setter`

Tuần 4



Kế thừa

`class Con(Cha), super(), override`

Tuần 5



Đa hình

5 dạng đa hình, ABC, Duck typing
Magic methods, Operator overloading

Tuần 6 ★

Bạn đã có đủ kiến thức OOP — từ tuần 7 sẽ áp dụng vào File I/O nâng cao, UML và Thiết kế!

Bài 1: Refactor Quản lý Hàng Hóa

Yêu cầu

Lấy lại bài tập Quản lý Hàng Hóa đã làm ở tuần 5 (HangHoa → HangDienMay, HangSanhSu, HangThucPham).

Refactor (viết lại) áp dụng ĐẦY ĐỦ các kiến thức đã học trong chương 6.

Giữ nguyên bài toán — thay đổi cách triển khai. So sánh code trước/sau để thấy sự khác biệt.

Kiến thức cần áp dụng

ABC + @abstractmethod

HangHoa là lớp trừu tượng, không tạo đối tượng trực tiếp.
loai_hang() và inTTin() là abstract methods.

@property

Getter cho ma_hang, ten_hang (read-only).
Getter + Setter cho gia (validation: giá >= 0).

Custom Exception

Tạo GiaKhongHopLe(Exception), MaHangTrungLap(Exception).
Raise trong @property setter và hàm thêm hàng.

Method Overriding (cùng tên)

Tất cả lớp con override inTTin() — CÙNG TÊN lớp cha (không dùng inTTin1).
→ Đa hình: for sp in ds: print(sp.inTTin())

__str__, __eq__, __lt__, __hash__

__str__: print(sp) hiển thị đẹp. __eq__: so sánh theo mã hàng.
__lt__: so sánh theo giá → sorted(). __hash__: dùng cho set().

Context Manager (with)

Dùng with open() để lưu và đọc danh sách hàng hóa từ file.

Bài 2: Nâng cấp bài Quản lý Cán bộ (chương 5)

Refactor bài cũ + áp dụng kiến thức chương 6



Đề bài gốc (chương 5)

Một đơn vị sản xuất gồm các cán bộ là công nhân, kỹ sư, nhân viên.

Thông tin chung: Họ tên, tuổi, giới tính, địa chỉ.

- Công nhân: thêm thuộc tính Bậc (1–10)
- Kỹ sư: thêm thuộc tính Ngành đào tạo
- Nhân viên: thêm thuộc tính Công việc

Yêu cầu (đã làm ở chương 5):

1. Xây dựng `CongNhan`, `KySu`, `NhanVien` kế thừa từ `CanBo`
2. Lớp `QLCB`: thêm mới, tìm kiếm theo tên, hiển thị danh sách, thoát

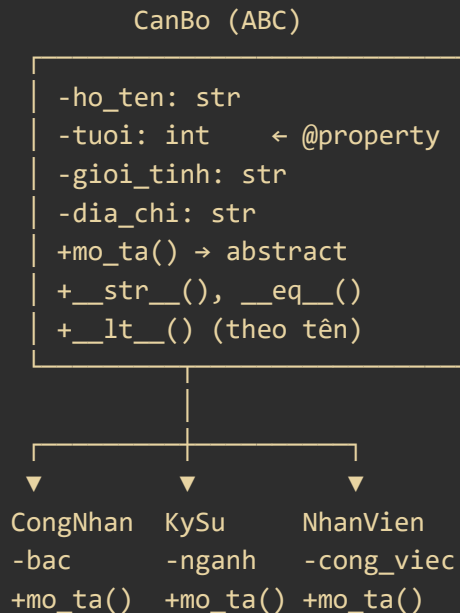


Yêu cầu bổ sung (chương 6)

Lấy lại code chương 5, NÂNG CẤP bằng cách áp dụng các kiến thức sau:

- ✓ ABC + `@abstractmethod`
CanBo là ABC, `mo_ta()` là abstract method
- ✓ Override cùng tên
Mỗi lớp con override `mo_ta()` → đa hình
- ✓ `@property` + validation
Tuổi (18–65), Bậc công nhân (1–10)
- ✓ `__str__`, `__repr__`
`print(cb)` hiển thị đầy đủ thông tin
- ✓ `__eq__` (theo họ tên + tuổi)
- ✓ `__lt__` (sắp xếp theo tên ABC)
- ✓ Custom exception
`TuoiKhongHopLe`, `BacKhongHopLe`
- ✓ with — lưu/đọc danh sách từ file

Bài 2: Nâng cấp bài Quản lý Cán bộ (chương 5)



Ví dụ đa hình – 1 vòng lặp, 3 loại CB:

```
ds = [CongNhan("Nguyen A", 30, "Nam",
              "HN", 5),
      KySu("Tran B", 28, "Nữ",
           "HCM", "CNTT"),
      NhanVien("Le C", 35, "Nam",
              "DN", "Kế toán")]
```

```
for cb in sorted(ds): # __lt__ → theo tên
    print(cb)         # __str__ → đa hình!
```

Tìm kiếm:

```
ten = input("Nhập tên: ")
kq = [cb for cb in ds
      if ten.lower() in str(cb).lower()]
```

Lưu file:

```
with open("canbo.txt", "w") as f:
    for cb in ds:
        f.write(repr(cb) + "\n")
```

Bài 3: Xây dựng lớp Phân Số

Operator Overloading + Magic Methods + @property + Exception



Yêu cầu đề bài

Xây dựng lớp PhanSo với hai thuộc tính riêng (private) xác định tử số và mẫu số của phân số.

Xây dựng các phương thức:

1. Hàm khởi tạo `__init__(self, tu, mau)`: kiểm tra mẫu số $\neq 0$, nếu sai raise `MauSoBangKhong`
2. Các phép toán `+`, `-`, `*`, `/` các phân số (dùng operator overloading: `__add__`, `__sub__`, `__mul__`, `__truediv__`)
3. Phép kiểm tra phân số có tối giản hay không (`is_toi_gian`)
4. Phép tìm dạng tối giản của phân số (`toi_gian`)
5. Phép so sánh `==`, `<`, `>` giữa các phân số (`__eq__`, `__lt__`, `__gt__`)
6. Hiển thị phân số đẹp: `__str__` trả về dạng "tử/mẫu" hoặc số nguyên nếu mẫu = 1

Viết chương trình ứng dụng:

- Nhập vào một dãy các phân số
- In ra màn hình dạng tối giản của các phân số đó
- Sắp xếp dãy phân số theo giá trị tăng dần (dùng `sorted()` nhờ `__lt__`)

Kiến thức chương 6 cần áp dụng

@property + validation

Getter/setter cho `tu_so`, `mau_so`.

Setter `mau_so`: raise `MauSoBangKhong` nếu = 0.

Operator Overloading

`__add__`, `__sub__`, `__mul__`, `__truediv__`

→ `ps1 + ps2`, `ps1 * ps2`... tự nhiên

`__eq__`, `__lt__`, `__gt__`

`ps1 == ps2` (so sánh giá trị thực)

`sorted(ds_ps)` → sắp xếp phân số

Custom Exception

`MauSoBangKhong`(Exception): ngoại lệ riêng khi mẫu số bằng 0.

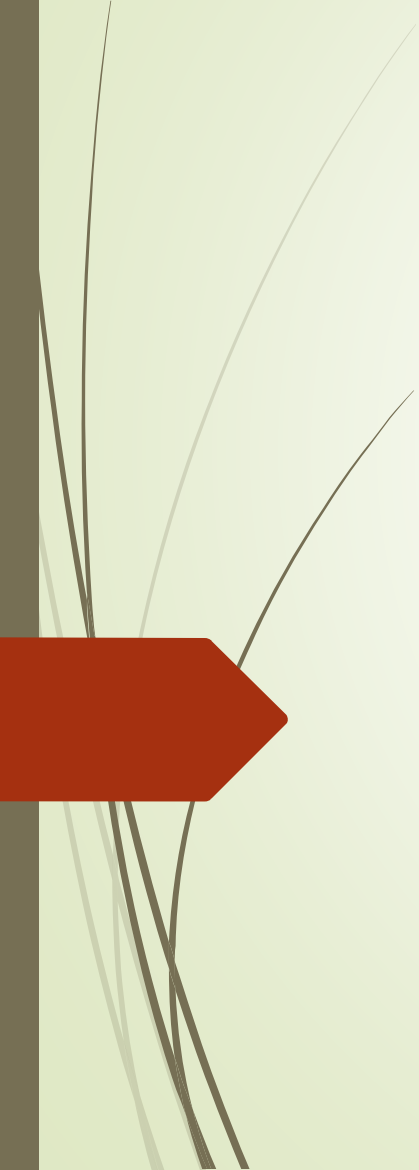
`__str__`, `__repr__`

`print(ps)` → "3/4" hoặc "2" (nếu mẫu=1)

`repr(ps)` → "PhanSo(3, 4)"

`__hash__`

Hash theo dạng tối giản → set() loại trùng phân số cùng giá trị (`2/4 == 1/2`)

A decorative graphic on the left side of the slide featuring several thin, curved lines representing grass or reeds. At the bottom left, there is a solid red arrow pointing to the right.

Thank you!

Any questions?